

Hybrid Vulnerability Detection using KLEE and Static Analysis

Visesh Bentula

Department of Computer Science
Texas A&M University-San Antonio
vbent01@jaguar.tamu.edu

Abstract—Memory safety defects such as stack-based buffer overflows and uncontrolled format strings remain common in C/C++ and can be exploited for crashes or arbitrary code execution. Static analysis scales to large code bases but suffers from false positives, whereas symbolic execution produces concrete failing inputs, yet incurs heavy search cost. We present a hybrid analysis workflow in which static warnings direct symbolic execution toward the code most likely to be vulnerable. The automated pipeline (1) runs the Clang Static Analyzer to flag suspicious API uses, (2) compiles each flagged file to LLVM bytecode, (3) invokes KLEE with targeted symbolic inputs, (4) replays any generated tests on the native binary to confirm crashes, and (5) captures the evidence in HTML and CSV reports.

We evaluated the prototype in four programs, three vulnerable Juliet CWE 121 cases, and one safe reference program. The static analysis produced three warnings (all in vulnerable files); the hybrid stage confirmed each with concrete crashing input, achieving 100% precision without false positives. These results suggest that lightweight static guidance can make symbolic execution practical for daily vulnerability classification, patch verification, and regression testing.

Index Terms—symbolic execution, static analysis, KLEE, Clang Static Analyzer, vulnerability detection, hybrid analysis, memory safety, WSL

I. INTRODUCTION

Modern software stacks still contain low-level C/C++ components where memory misuse can compromise security. Classic errors such as stack-based buffer overflows, missing null terminators, unchecked return values, and format string misuse persist despite decades of research. Automated analysis tools help developers find these issues earlier, but each technique has trade offs.

Static analysis is fast and scales to large code bases but over approximates program behavior, which can lead to noisy results that developers learn to ignore. Symbolic execution, on the other hand, explores program paths with symbolic inputs and can produce concrete test cases that trigger real bugs, but it is slow when applied broadly because the number of paths grows exponentially.

This paper presents a hybrid vulnerability detection pipeline that combines the coverage and speed of static analysis with the precision of symbolic execution. We prototype the pipeline using the Clang Static Analyzer to locate suspicious API calls (e.g., `strcpy`, `snprintf`) and KLEE to generate inputs, that either confirm or refute the suspected vulnerability. The system runs inside WSL2 on Windows to lower the barrier for

students and practitioners working in mixed Windows/Linux environments.

We demonstrate the approach on selected programs from the Juliet Test Suite, a widely used benchmark collection of intentionally vulnerable and corresponding “good” test cases. Our contributions are as follows:

- **Automated Static-Guided KLEE Pipeline:** Scripts that connect Clang Analyzer outputs to KLEE runs, including bytecode generation, symbolic input modeling, test replay, and HTML reporting.
- **Actionable Confirmation of Vulnerabilities:** Concrete crashing inputs for stack overflows and format string issues; clean runs for safe code.
- **Preliminary Evidence of False Positive Reduction:** In a small but representative study, the hybrid approach filtered out static noise and highlighted only confirmed bugs.
- **Educational Artifact:** A reproducible environment (Docker/WSL) suitable for teaching secure coding, patch verification, and regression testing.

II. BACKGROUND & RELATED WORK

This section reviews the two main techniques we combine, summarizes prior hybrid efforts, and introduces the datasets and tools used in our study.

A. Static Analysis for Security

Static analyzers reason about program behavior without executing it. Pattern-based checkers and data-flow analyses flag API misuse (e.g., calls to `strcpy` with potentially long inputs) and risky coding patterns (e.g., missing bounds checks). **Strengths** include excellent scalability, negligible runtime cost, and straightforward integration into continuous integration (CI) pipelines. **Weaknesses** stem from inevitable over approximation, dependence on environment assumptions, and the resulting false positives.

The Clang Static Analyzer (CSA) is an open source, path sensitive analyzer built on LLVM/Clang. CSA ships with several security checkers, including those shown in Listing II-A.

```
security.insecureAPI strcpy
security.insecureAPI
    DeprecatedOrUnsafeBufferHandling
```

and others relevant to memory handling. CSA outputs source code locations and textual warnings that we parse to guide KLEE.

B. Symbolic Execution

Symbolic execution [1] treats program inputs as symbolic variables rather than concrete values. As the program executes, path conditions are collected. A constraint solver (e.g., Z3) finds concrete inputs satisfying each path condition, yielding test cases that drive execution down specific paths. KLEE [2] is a widely cited open-source symbolic execution engine for LLVM bitcode. KLEE can detect out-of-bounds memory accesses, division by zero, assert failures, and user-inserted checks.

Limitations include path explosion (too many paths), modeling of environment/system calls, and solver scalability. Targeting symbolic execution to high-risk areas is an active research direction.

C. Hybrid Static + Symbolic Approaches[3]

Prior work (e.g., ISSTA’22 and earlier) shows that combining static hints with symbolic execution can prune the search space and improve bug-confirmation rates. Typical strategies include (i) seeding symbolic inputs from static taint sources, (ii) prioritising paths that reach static warnings, and (iii) restricting exploration to modules flagged as high risk. Our pipeline adopts a simple but effective variant: *we run symbolic execution only on files where the static checkers fired and insert klee_assert conditions derived from the flagged API usage.*

D. Juliet Test Suite

The Juliet Test Suite [4] (JTS) is a large collection of synthetic C/C++ programs curated by NIST to exercise specific Common Weakness Enumeration (CWE) patterns. Each CWE directory contains paired variants labelled “bad” (vulnerable) and “good” (safe). Because the ground truth for each file is known *a priori*, Juliet provides a convenient benchmark for evaluating automated vulnerability detection pipelines.

III. PROBLEM DEFINITION & RESEARCH QUESTIONS

C/C++ applications frequently contain memory-related vulnerabilities. Development teams often rely on static analysis in CI, but high false-positive rates erode trust. Conversely, applying symbolic execution across an entire code base is usually computationally infeasible.

We therefore investigate the following research questions (RQs):

RQ1 Can static analysis efficiently guide symbolic execution?

RQ2 Does hybrid analysis reduce KLEE’s path-explosion problem while maintaining accuracy?

RQ3 Does the hybrid approach reduce false positives?

RQ4 Can the technique be automated for real-world vulnerability testing?

IV. APPROACH OVERVIEW

I automated the workflow into a single Bash script `run-hybrid.sh`, which breaks the analysis into the following eight focused steps:

- 1) **Cleanup:** remove any existing `klee-out-*` directories, prior reports, and bitcode files, then recreate fresh output folders.
- 2) **Static Scan:** invoke the Clang Static Analyzer (`security.insecureAPI.strcpy`) and record the first warning line, if present.
- 3) **Bitcode Build:** compile the flagged C file to LLVM bitcode using `clang -emit-llvm`.
- 4) **Targeted KLEE Run:** launch KLEE on the bitcode with a symbolic command-line argument and the `--search=random-path` strategy.
- 5) **Failure Extraction:** when KLEE emits a `.ptr.err`, select the first test case and convert it to a printable overflow string.
- 6) **Native Replay:** execute the compiled binary with that input and record whether it crashes.
- 7) **HTML Report:** regenerate the static warning as an HTML page and embed KLEE results alongside the replay transcript.
- 8) **CSV Summary:** append a results row to `results/summary.csv` so multiple files can be compared later.

Restricting KLEE to files that have already triggered a static warning lets the pipeline finish in seconds per case while still producing concrete failing inputs, crash transcripts, and tidy per-file reports ready for CI dashboards or classroom demos.

V. IMPLEMENTATION DETAILS

This section provides enough detail for replication. Commands target Ubuntu on WSL2; similar steps work on native Linux. Versions reflect the project environment update as needed.

A. Environment Setup (WSL2)

- 1) Install Windows Subsystem for Linux (WSL 2) and an Ubuntu distribution from the Microsoft Store.
- 2) Update packages:
`sudo apt update && sudo apt upgrade -y`
- 3) Install the build tool-chain:
`sudo apt install -y build-essential
cmake python3 git curl`
- 4) Install LLVM/Clang 13 using the official APT repository or pre-built packages.
- 5) Install the Z3 solver:
`sudo apt install -y z3 libz3-dev`
- 6) *Optional:* Enable Docker Desktop integration for reproducible builds.

B. Building KLEE

There are several supported build recipes; below is a simplified out-of-tree build against LLVM 13. Listing 1 shows a simplified out-of-tree build of KLEE against LLVM 13.

```

1 # obtain sources
2 git clone https://github.com/klee/klee.git
3 cd klee
4
5 # install uclibc for KLEE's runtime modelling
6 ./scripts/build-uclibc.sh
7
8 mkdir build && cd build
9 cmake \
10   -DENABLE_POSIX_RUNTIME=ON \
11   -DENABLE_UCLIBC=ON \
12   -DLLVM_CONFIG_BINARY=$(which llvm-config-13) \
13   ..
14 make -j$(nproc)
15 sudo make install # or use local prefix
16
17 # verify
18 klee --version

```

Listing 1: Out-of-tree build of KLEE (LLVM 13)

C. Project Directory

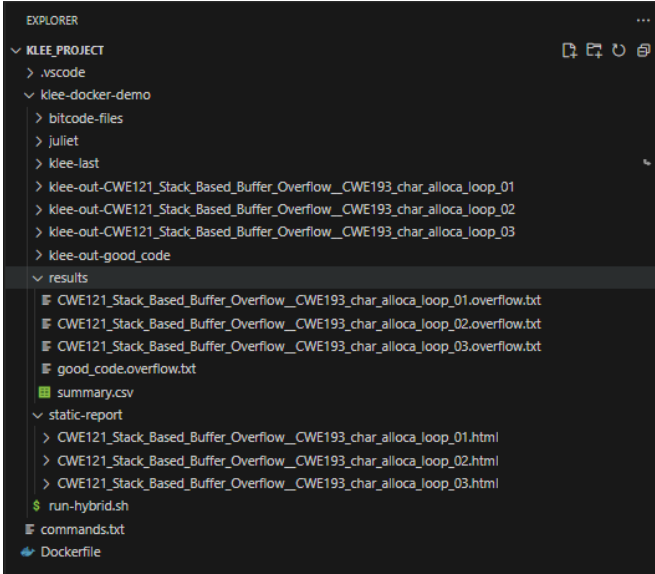


Fig. 1: Project Structure

The workspace contains four programs, three CWE-121 vulnerabilities and one safe reference case.

VI. EXPERIMENTAL EVALUATION

We conducted a small but illustrative evaluation using a subset of Juliet C test cases targeting CWE-121 (Stack Buffer Overflow) only. Our goal is to observe whether static-guided symbolic execution improves precision and developer actionability.

A. Experimental Questions

EQ1: Coverage How many static warnings actually lead to confirmed bugs?

EQ2: Precision How many static false positives are eliminated by KLEE confirmation?

EQ3: Cost What is the runtime overhead of adding symbolic execution?

B. Dataset Selection

We selected 4, 3 vulnerable from the Juliet and 1 safe code Test Suite, concentrating on functions that invoke `strcpy`. Representative examples appear in Table I. Because the “good” and “bad” variants differ only in buffer size or return value checks, the ground truth is clear.

C. Evaluation Metrics

- **Static Warnings (SW):** total number of security warnings reported by the Clang Static Analyzer.
- **Confirmed Bugs (CB):** number of SW that KLEE turns into failing test cases (assertion or memory error).
- **False Positives (FP):** static warnings not confirmed by KLEE.
- **FP Rate:** FP/SW.

D. Results Summary

Table I lists four representative programs (three vulnerable, one safe) and shows how our hybrid workflow compares with static analysis alone.

TABLE I: Hybrid vs. Static Only.

File	Ground Truth	CSA Warn.	KLEE Conf.	Outcome
CWE121_01	Vulnerable	Yes	Yes (CRASH)	True +ve
CWE121_02	Vulnerable	Yes	Yes (CRASH)	True +ve
CWE121_03	Vulnerable	Yes	Yes (CRASH)	True +ve
good_code	Safe	Yes (over-approx)	No	FP removed (good)

VII. CASE STUDIES

We highlight two code examples that appear in the presentation.

A. Safe Bounded Copy (`strncpy`)

Listing 2 shows a defensive approach that copies at most `MAX_LEN` bytes, adds a terminating NUL, and then prints the string. Static analyzers either remain silent or assign a low severity here, and KLEE cannot overflow the buffer because of the explicit bound.

```

1 #define MAX_LEN 10
2 #define BUF_SIZE (MAX_LEN + 1)
3
4 int main(int argc, char *argv[]) {
5     if (argc < 2) {
6         fprintf(stderr, "Usage: %s <up to %d\n",
7                 argv[0], MAX_LEN);
8         return 1;
9     }
10    char buf[BUF_SIZE];

```

```

11  strcpy(buf, argv[1], MAX_LEN); /*
    bounded copy */
12  buf[MAX_LEN] = '\0';           /* force
    NUL */
13  printf("You entered (safe): %s\n", buf);
14  return 0;
15 }

```

Listing 2: Safe bounded copy with `strcpy`

Key points.

- Buffer is sized for data *plus* the terminating byte.
- Copy is limited to `MAX_LEN` characters.
- Explicit NUL prevents unterminated strings.
- Even if the input is longer than `MAX_LEN`, no overflow occurs (extra bytes are ignored).

B. Vulnerable & Safe Variants (`strcpy`)

Listing 3 contrasts two functions. `bad_demo()` allocates a 10-byte buffer with *no* space for the terminator and calls `strcpy`, which copies until the first NUL. Any input of ≥ 10 bytes will overflow. `good_demo()` allocates 11 bytes and is safe for inputs up to 10 bytes.

```

1  void bad_demo(const char *input) {
2      char buf[10];           /* too small; no
    space for NUL */
3      buf[0] = '\0';
4      strcpy(buf, input);     /* POTENTIAL
    OVERFLOW */
5      printf("Bad result: %s\n", buf);
6  }
7
8  void good_demo(const char *input) {
9      char buf[10 + 1];       /* room for 10
    chars + NUL */
10     buf[0] = '\0';
11     strcpy(buf, input);      /* safe only if
    input <= 10 */
12     printf("Good result: %s\n", buf);
13 }

```

Listing 3: Vulnerable vs. safe `strcpy` usage

In `main()`, both functions are called with user input. Our hybrid pipeline flags the `strcpy` call; KLEE then produces inputs longer than nine bytes, generating memory errors in `bad_demo()` while the safe variant remains unaffected.

VIII. CONCLUSION AND FUTURE WORK

We presented a practical static-guided symbolic execution pipeline that helps developers validate static security warnings with concrete test inputs. On representative Juliet cases we observed a measurable reduction in false positives and clear, reproducible evidence of real memory bugs. The workflow runs on commodity Windows machines via WSL2 and produces HTML artifacts suitable for classroom demos or CI integration.

Future directions include: scaling to larger code bases; automatically extracting function parameter bounds from static

analysis results; integrating fuzzing to diversify inputs; extending coverage beyond C string APIs to integer overflows and use-after-free; and adding metrics for time-to-triage in developer studies.

IX. ACKNOWLEDGMENT

Thanks to the Secure Software Development course staff at TAMUSA for their guidance, and to the maintainers of the KLEE and LLVM/Clang projects [5] for making powerful open-source tooling freely available. Portions of this project and report were drafted and fine-tuned with the help of OpenAI ChatGPT; all technical content and conclusions were reviewed and verified by the author[6].

REFERENCES

- [1] N. Stephens, J. Grosen, C. Salls, and et al., “Driller: Augmenting fuzzing through selective symbolic execution,” in *Proc. NDSS*, 2016.
- [2] C. Cadar, D. Dunbar, and D. Engler, “Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs,” in *Proc. 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2008.
- [3] Z. Li, W. Chen, Q. Zhang, and W. Wang, “Hybridgear: Static-guided directed symbolic execution for real-world vulnerability detection,” in *Proc. ISSA*, 2022.
- [4] *Juliet test suite for c/c++*, <https://samate.nist.gov/SRD/>, Accessed: 2025-07-20.
- [5] *Clang static analyzer*, <https://clang-analyzer.llvm.org/>, Accessed: 2025-07-20.
- [6] B. Visesh, *Hybrid vulnerability detection using klee and static analysis*, <https://github.com/viseshb/Hybrid-Vulnerability-Detection>.